

The 9th International Workshop on Computational Antifragility and Antifragile Engineering
March 22-25, 2022, Porto, Portugal

Residuality Theory, random simulation, and attractor networks

Barry M O'Reilly*

*Black Tulip Technology
Department of Engineering and Innovation, Open University Milton Keynes UK*

Abstract

This article reviews earlier articles on the topic of residuality theory and places residuality theory in the context of the complexity sciences, relating the major concepts of residuality theory in terms of Kauffman Networks, complex networks, random simulations, and attractors. This paper aims to outline the logic of the theory and to expand on the use of matrices for encouraging emergent component decomposition, at the same time as proposing a related theory of software engineering that allows comparison between residuality theory and other approaches.

© 2022 The Authors. Published by Elsevier B.V.

This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>)

Peer-review under responsibility of the Conference Program Chairs.

Keywords: Residuality; antifragility; resilience; software engineering

1. Introduction

Despite enormous growth in the last few decades, there is still no thorough theoretical grasp of how software engineering actually happens. The empirical study of software engineering is made extremely difficult by the idiographic nature of software projects. The experiences of software engineers are difficult to capture, vary wildly, and are most often reported rather than observed since the crucial decision making in software projects happens in the minds of practitioners. Decision making in software is arbitrary and difficult for practitioners to explain [1]. Whilst there are many methods, frameworks, and ideas describing how this process could be carried out, no two

* Corresponding author..

E-mail address: barry@blacktulip.se

projects can be shown to follow the exact same trajectories.

Despite this predicament, many software architects consistently deliver quality architectures across long careers. How this is achieved is not easily captured by frameworks, textbooks, or methodologies. Architecture is still learned through experience. The most prevalent architectural certifications are experience based rather than exam based. Even when taught specific tools or claiming to use particular frameworks, most software engineers are uncertain about how or why they actually make decisions [1]. This leads to the conclusion that most academic and industry theories of software engineering are based on practices that do not reflect the real world of software engineering. There are so many heuristics and differing narratives that mapping and describing the work of software engineers is extremely difficult. For example, Donald A Schön's [3] description of reflective practice is reminiscent of the approaches of software architects, whilst descriptions of most traditional approaches to software engineering would fall under the label of technical rationality described by Schön. This creates a gap between theory and practice that is difficult to bridge. This paper attempts to bridge that gap by describing the process of software design as a simple two-step algorithm that should be present in most approaches, reflective and otherwise.

This article will summarize the conceptual papers on residuality theory written between 2019-2021[4][5][6][7][8] through the lens of this simple algorithm. Furthermore, it will show the journey from observation to theory and outline the future research path on the subject. Finally, the paper aims to present residuality theory as a fully-fledged theory that can be related to the actual practices of software engineers and provide a basis for further empirical investigation.

1.1. Building Residuality Theory as a Theory.

A theory consists of constructs, propositions, and logic [3]. Previous papers [4][5][6][7][8] have outlined the observed behavior and described constructs and propositions. This paper will summarize some of these. Furthermore, it will focus on providing the underlying logic for the theory – an explanation of why the theory works.

The original 2019 paper [4] described a number of heuristics observed in practice to impact the quality of software architectures. It linked Taleb's observation of the potential positive impact of stress on structures to architectural decision making.

A strange correlation was observed between using this set of heuristics and the ability of systems designed to survive stressors – events for which they had not been designed. By abandoning traditional requirements engineering and risk analysis methods, but instead using randomized stressors to drive design, it eventually became challenging to stress the system. When this was combined with simple analyses using matrices, it allowed a software structure to emerge, which appeared to be better at surviving unknown forms of stress than alternative approaches. Moreover, these systems appeared to have a high level of quality.

By 2020 the ideas were placed into a theoretical frame called residuality theory, influenced by further reading in the complexity sciences [5]. The cited article defined several important constructs that provided a foundation for understanding and replicating the observed behaviors in the 2019 paper. Subsequent papers described the philosophical difficulties in unifying residuality theory and other software engineering methodologies [6][7]. The 2021 paper on hyperliminal coupling [8] laid the foundation for describing software architects' central problem, which residuality theory aims to address. The success of a software application depends on the software's performance under conditions for which it has not been designed. In dynamic, ever-changing environments where no scientific foundation for prediction exists, architects are forced to make predictions as grounds for structural decisions in software. These structural decisions introduce coupling that may constrain the future choices of the organization at a software and business level.

Constructs

1.2. Stressors

When architects work with a software system, they are forced to work toward an unknown future. Any event that arises in that unknown future, which the system is not designed for, is known as a stressor.

1.3. Hyperliminality

Hyperliminality describes an ordered system inside a disordered system [8]. An ordered system is predictable, mappable, and testable. A disordered system is dynamic, growing, and unpredictable. The architect is forced to constantly move between these two worlds, with ordered software and disordered enterprise contexts which require entirely different tools and epistemologies to understand.

1.4. Hyperliminal Coupling

If two nodes in a network each have a relationship with a third node, then those two nodes are very likely to have a relationship. Therefore, if a stressor in the wider hyperliminal system interacts with two software components, then those two components can be considered coupled. Since architects are unaware of the stressor, this coupling is invisible to the system's designer until the stressor is realized. This is called hyperliminal coupling and provides some explanation as to the difficulty of software engineering.

1.5. Residues

When a stressor impacts the system, it is necessary to isolate the impacted and relevant nodes and assess how the network can be improved to handle the transition to a potential new state. This subset of nodes under consideration is called the residue, and the work involved means augmenting the residue – changing its structure - to make the transition easier. The states that a system visits most often are described as attractors. Residues act as a container that allows the architect to view the entire system through the lens of many different stressors and attractor transitions happening at different times in different orders before committing to a specific architecture. This abstraction makes the mental work of thinking through possible futures easier.

1.6. Residuality Theory

Residuality theory is a minimalistic description of the software engineer's world. It is an epistemological position on what can be known in the software engineer's view and a reasonably pessimistic one. Residuality theory applied to software engineering gives rise to a framework called residual analysis [5]. Residuality theory makes three simple statements:

- 1) Enterprise software systems are ordered systems that live in disordered environments - hyperliminal systems.
- 2) These systems will experience stress that they have not been designed for because the disordered environment is by definition unpredictable.
- 3) The system's future is a function of residue, whatever is left over after it is stressed.

2. Proposition

Software engineering in hyperliminal environments can be described as a combination of a random simulation of the environment followed by the analysis of the software's architectural structure as a network in the context of the environment. Residual analysis makes this simulation more random and wide-reaching and the network analysis more explicit than traditional software design methods. Furthermore, by adopting the ideas behind residuality theory, modeling residues instead of components directly, it is possible to identify hyperliminal coupling in the design phase and produce software structures better placed to withstand unknown sources of stress.

2.1. Boundaries

Residuality theory is concerned with bespoke enterprise software development. However, other researchers are working with these ideas in organizational design and that is considered a separate research area.

2.2. Falsifiability

The experiment described in section 5 will show that residuality theory can be falsified at the level of individual software applications.

3. Theoretical Framework

In order to discuss and place residuality theory in context, it is necessary to introduce several ideas from the complexity sciences.

3.1. Random Simulation.

In hyperliminal environments where there is a significant flux in the disordered, non-software part of the system, it is very difficult for software engineers to identify requirements or risks precisely. Each future state of the hyperliminal system that cannot be predicted becomes a potential stressor for the application, built according to a minimal subset of the potential system states. Given the number of variables and states in the hyperliminal system, it is impossible to work through them all. Thus, many software engineering methodologies use heuristics and techniques to reduce the size of the state-space investigated. Requirements engineering focuses on explicit, correct, and precise statements about the system, reducing uncertainty about what should be built. Risk management techniques use the filter of probability and impact to make the number of considered possibilities fewer. Given that software applications consistently experience stressors, reducing the number of such events considered seems contrary to the aims of the software designer. Most traditional methods ignore uncertainty by placing responsibility on the requirements engineering and risk management processes, and the design of component structures is based unquestioningly on the results of these.

Nevertheless, those architects who succeed do not explore the entire state space. In other fields, such as statistical mechanics or finance, random simulations such as Monte Carlo simulations are used. By choosing random variables and processing them, these simulations can cover enough of the state space to allow for convergence toward an approximate result. The state-space of hyperliminal software systems is so large as to be mathematically intractable, and no probability distribution exists with which to work. However, adapted simulations such as Fuzzy Monte Carlo distributions have been used in situations where possibilities exist without knowing the probabilities of these events, so the practice of random simulation can extend beyond the mathematically feasible. In effect, the conclusions of requirements engineering and risk analyses in software are already random simulations, educated guesses driven by

experience and bias, given that stakeholders do not know the future. Traditional methods in software architecture viewed as random simulations would seem to suffer from the curse of dimensionality – distributions tend to return to areas of high probability. Many of the ideas in stressor analysis[5] – the use of random nonsense stressors and impossible events – combat this instinctively.

Enhancing this already randomized simulation with random stressors, increasing both randomness and sample size, should lead to better identification of hyperliminal coupling and, therefore, better approximations of resilient architectural structures. As it is not feasible (and possibly damaging to try) to accurately describe hyperliminal environments mathematically, the simulation is carried out by collecting narratives. This approach has developed as a heuristic and is described in [4][5]. The following section will demonstrate how the properties of complex networks and hyperliminality aid the software designer in making this random simulation converge faster than it otherwise would.

3.2. *Kauffman Networks*

Kauffman networks [9] refer to Random Boolean Networks first described by Kauffman in 1969. Kauffman used these networks to demonstrate the presence of attractors – a limited number of states in the network's state space to which the system will repeatedly return. Kauffman describes three variables and their relationship to order or disorder in these networks.

N is the number of nodes in the system.

K is the highest number of connections a node has in the network.

P is a node's bias toward delivering a particular result when processing a signal.

Increasing N and K leads to a much greater number of attractors. Increasing P for nodes leads to fewer outcomes and is thus easier to manage.

3.3. *Complex Networks, adjacency matrices, and incidence matrices.*

Complex networks are used to represent nontrivial, multi-agent multi-level networks. They follow similar patterns to those of Kauffman networks. It is these that we will use to model the world of software. As mentioned earlier, software is an ordered network, whereas the wider business network is disordered, dynamic, and growing. Software is considered a network of residues, which are made up of networks of components, which are made up of networks of functions, and these functions are linked by information flows between nodes in the hyperliminal network. However, these networks can be added to and changed depending on the context. The application of the ideas in Kauffman networks to these kinds of systems can be seen in Marion [10] and [11].

In software engineering, N refers to components, K describes connections or calls between them. P is increased by using contracts, schemas, policies and reducing the number of branches in code. For this article, this kind of analysis on software is called an NKP analysis. It is not difficult to see how vague software engineering concerns such as loose coupling, cohesion, and component granularity can be expressed more clearly in terms of NKP analysis. NKP analysis is only applied to the software part of the network, although occasionally it has been pertinent to consider some nodes from the wider environment to build understanding.

By manipulating NKP, it is possible to present new networks, and hence new architectures, that are more or less able to cope with the generated stress. Moreover, Kauffman speaks of the right balance of N, K, and P pushing the network to the edge of chaos – a point of stability that also offers affordances, the ability to move between attractors. This idea suggests that a network too optimized for order and low numbers of attractors may be vulnerable to unexpected changes. A network too disordered will offer too many attractors and be easily affected by external stress.

Analyzing the component structure as a network involves producing matrices and using simple heuristics to make decisions about groupings of functions. Two types of matrices are necessary: adjacency matrices and incidence matrices. This allows for the simple recognition of patterns that cause difficult NKP configurations and makes it easier to visualize hyperliminal coupling. Some examples are given below.

3.3.1. Adjacency matrices

These are directed matrices used to investigate dependency between nodes of the same type. Processes, information flows, software functions, and software components are typically analyzed in these matrices. Careful attention is paid to symmetry in these matrices, representing multidirectional coupling and opportunities for stressors to spread through the architecture. Given the focus on dependency, these adjacency matrices can be combined and have a lot in common with Design Structure Matrices. These matrices help identify unnecessary links and opportunities to combine interdependent nodes, thus decreasing N and K and increasing P.

3.3.2. Incidence matrices

Incidence matrices map stressors against residues, processes, flows, software functions, and software components. These analyses allow the architect to clearly visualize the most vulnerable components and the most dangerous stressors. Furthermore, the components that show similar patterns in the matrices can be grouped together as this indicates hyperliminal coupling between them. This, combined with the identified dependencies in 4.3.1, provides much information on hyperliminal coupling in the application and allows the architect to refactor accordingly.

4. Residual analysis: random simulation combined with network analysis.

This approach becomes even more useful because of the existence of attractors in the software structure. Given that the business environment has a much greater number of nodes, it is known that the number of attractors, or phase states, is dramatically greater than the number of phase states in the software structure [12] [13]. Thus, as it increases in size, a random selection of stressors should revisit many of the same attractors in the software system. Thus, even an irrelevant stressor with almost zero probability may point to an attractor that could be required to survive a completely different, less visible stressor. If this holds, then an extended random simulation should force changes in the software structure that eventually begin to absorb other stressors to which the system is exposed. Random stressors, therefore, reveal the unknown K, hyperliminal coupling, which, when exposed and resolved through matrix analysis, allows the software architecture to be refactored to cope with the increase in attractors this previously unknown K introduces. This has been the observed behavior in multiple software projects, and therefore this theory appears to have enough explanatory power to describe the observed phenomenon.

To show that this has worked, the test mentioned in 2020 [5] can be used. Dividing the stressors into a training and testing set and using the testing set after the random simulation and network analysis are performed, it is easy to see empirically and directly if the methodology has worked, using the original naïve architecture as a control. This allows the calculation of the residual index Ri.

The value of Ri can be calculated thus:

Let S be the total number of stressors

Let N be the number of stressors survived by the naïve architecture

Let R be the number of stressors survived by the new, residual architecture

In cases where neither architecture survives a particular stressor, increment N or R by 1 based on which architecture best survives the stressor.

Then $R_i = \frac{R-N}{S}$ giving $-1 \leq R_i \leq 1$

Therefore, a residual index greater than 0 represents a successful intervention and has produced a system more likely to survive unknown forms of stress than the naïve architecture. For example, a recent lab run for students produced residual index values of 0.27 – 0.57 with a few hours of hurried lab work (this experiment, however, was not run under stringent empirical conditions and does not represent proof of efficiency, merely an anecdotal example of a consistently observed pattern that should provoke further empirical inquiry). Further iterations should push this number much higher. Given the idiographic nature of software, it is crucial that this test can be run independently across software projects, even if the theory is shown to be widely generalizable.

The two-step algorithm of random stressor simulation followed by NKP network analysis can be shown to exist implicitly in any software design methodology. This will make it easier to compare software engineering methodologies with residuality theory and discuss how other methodologies handle uncertainty in comparison. Whilst it is possible to express this two-step algorithm in mathematical terms, it is considered alienating for those without a mathematical background and for now will continue to be expressed in terms of practical approaches in order to reach the widest possible audience of practitioners.

5. Conclusion

Residuality theory is a grounded theory that provides constructs, propositions, and logic. It allows the architect to explore that system as an attractor network exposed to a largely unknown environment instead of a response to a requirements engineering and risk analysis approach that ignores uncertainty in the environment. It clarifies the kind of thought experiments architects can use to navigate the mathematically intractable uncertainty that defines software projects. This article has shown that the theory's proposition holds. Software design can be described as a random simulation followed by a network analysis. Residuality theory makes explicit and amplifies these two steps, and can thus improve the ability of the system to withstand unknown stressors and increase quality. This is testable by experiment in every case. Future research is planned to provide empirical validation through experiment on a broader scope.

References

- [1] Ralph, P.; Tempero, (2016) E. Characteristics of decision-making during coding. In Proceedings of the 20th International Conference on Evaluation and Assessment in Software Engineering, Limerick, Ireland, 1–3 June 2016; pp. 1–10.
- [2] Schön, D. A. (2017). *The reflective practitioner: How professionals think in action*. Routledge
- [3] Bhattacharjee, A. (2012). *Social science research: Principles, methods, and practices*. University of South Florida.
- [4] O'Reilly, B.M (2019) No More Snake Oil: Architecting Agility through Antifragility *Procedia Computer Science*, 151 (2019), pp. 884-890
2019, ISSN 1877-0509, <https://doi.org/10.1016/j.procs.2019.04.122>.
- [5] O'Reilly, B.M. (2020) An introduction to residuality theory: Software design heuristics for complex systems. *Procedia Comput. Sci.* 170, 875–880.
- [6] O'Reilly, B.M. (2021) The Philosophy of Residuality Theory. *Procedia Comput. Sci.*, 184, 809–816.
- [7] O'Reilly BM. (2021) The Machine in the Ghost: Autonomy, Hyperconnectivity, and Residual Causality. *Philosophies*. 2021; 6(4):81. <https://doi.org/10.3390/philosophies6040081>
- [8] O'Reilly, B.M. Hyperliminal Coupling, (2021) Why Software Projects Fail Repeatedly; Cutter Consortium: Arlington, MA, USA,
- [9] Kauffman, S. A. (1993). *The origins of order: Self-organization and selection in evolution*. Oxford University Press, USA.
- [10] Marion, R. (1999). *The edge of organization: Chaos and complexity theories of formal social systems*. Sage.
- [11] Wuensche, A. (2004). Basins of Attraction in Network Dynamics: A Conceptual Framework for Biomolecular Networks chapter 13. *Modularity in development and evolution*, 288-311
- [12] Samuelsson, B., & Troein, C. (2003). Superpolynomial growth in the number of attractors in Kauffman networks. *Physical Review Letters*, 90(9), 098701.
- [13] U. Bastolla, G. Parisi, (1998) The modular structure of Kauffman networks, *Physica D: Nonlinear Phenomena*, Volume 115, Issues 3–4, 1998, Pages 219-233, ISSN 0167-2789, [https://doi.org/10.1016/S0167-2789\(97\)00242-X](https://doi.org/10.1016/S0167-2789(97)00242-X)